

# **Laporan Tugas Pembelajaran Mesin**



Dibuat Oleh:

Muhammad Naufal Zaki | 187241115

Parikesit Putra Pratomo | 187241053

SISTEM INFORMASI  
FAKULTAS SAINS DAN TEKNOLOGI  
UNIVERSITAS AIRLANGGA

# **BAB I PENDAHULUAN**

## **Latar Belakang**

Pasar mobil bekas di India merupakan salah satu pasar otomotif terbesar di dunia dengan volume transaksi yang sangat tinggi setiap tahunnya. India memiliki populasi lebih dari 1,4 miliar jiwa dan industri otomotifnya berkembang pesat, menjadikan mobil bekas sebagai pilihan utama bagi mayoritas masyarakat yang menginginkan kendaraan pribadi dengan harga lebih terjangkau. Namun demikian, harga mobil bekas sangat bervariasi tergantung pada berbagai faktor seperti tahun produksi, jarak tempuh, jenis bahan bakar, tipe transmisi, kapasitas mesin, tenaga maksimum, dan status kepemilikan. Keragaman faktor-faktor ini menyebabkan kesulitan bagi pembeli dan penjual dalam menentukan harga yang adil dan wajar.

Dalam konteks ini, prediksi harga otomatis menggunakan teknologi Machine Learning menjadi sangat penting untuk membantu para pembeli dan penjual mobil bekas mendapatkan estimasi harga yang objektif dan konsisten. Sistem prediksi berbasis Machine Learning dapat menganalisis pola historis dari data transaksi mobil bekas dan menghasilkan estimasi harga yang mendekati harga pasar sebenarnya. Hal ini tidak hanya menguntungkan konsumen, tetapi juga dapat digunakan oleh dealer mobil, platform jual-beli online, dan lembaga pembiayaan dalam pengambilan keputusan bisnis.

Pada proyek ini, dipilih metode Ensemble Learning berbasis Gradient Boosting, yaitu XGBoost (eXtreme Gradient Boosting) dan LightGBM (Light Gradient Boosting Machine). Kedua algoritma ini merupakan state-of-the-art untuk data tabular dan telah terbukti secara luas dalam berbagai kompetisi Machine Learning serta aplikasi industri. XGBoost dan LightGBM memiliki kemampuan untuk menangani data heterogen, missing values, dan fitur non-linear dengan sangat baik, menjadikannya pilihan yang tepat untuk dataset mobil bekas yang memiliki karakteristik demikian.

Tujuan akhir dari proyek ini adalah membangun model prediksi harga mobil bekas dengan akurasi tinggi serta melakukan deployment melalui aplikasi web interaktif menggunakan Streamlit. Aplikasi yang diberi nama CarPrice AI ini memungkinkan pengguna untuk memasukkan spesifikasi mobil dan mendapatkan estimasi harga secara real-time berdasarkan

model terbaik yang telah dilatih.

## **Rumusan Masalah**

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam proyek ini adalah sebagai berikut:

1. Bagaimana membangun model Machine Learning yang mampu memprediksi harga jual mobil bekas dengan akurasi tinggi menggunakan algoritma XGBoost dan LightGBM?
2. Fitur-fitur apa saja yang paling berpengaruh dalam menentukan harga jual mobil bekas?
3. Model mana yang memberikan performa terbaik antara XGBoost dan LightGBM berdasarkan metrik evaluasi regresi?
4. Bagaimana cara mendeploy model terbaik ke dalam aplikasi web interaktif yang dapat digunakan oleh masyarakat umum?

## **Tujuan Penelitian**

Tujuan yang ingin dicapai dalam proyek ini adalah sebagai berikut:

- Membangun dan melatih model prediksi harga mobil bekas menggunakan algoritma XGBoost dan LightGBM.
- Menganalisis fitur-fitur yang paling signifikan dalam memprediksi harga jual mobil bekas.
- Membandingkan performa kedua model berdasarkan metrik evaluasi regresi (MAE, MSE, RMSE, MAPE, dan R<sup>2</sup>).
- Mengembangkan aplikasi web interaktif berbasis Streamlit untuk deployment model prediksi harga mobil bekas.

## **BAB II METODE YANG DIPAKAI**

### **Ensemble Learning**

Ensemble Learning merupakan teknik dalam Machine Learning yang menggabungkan prediksi dari beberapa model individu untuk menghasilkan prediksi akhir yang lebih baik dan lebih stabil. Ide dasar dari ensemble learning adalah bahwa kelompok model yang bekerja bersama dapat menghasilkan performa yang lebih superior dibandingkan dengan model tunggal. Terdapat beberapa pendekatan dalam ensemble learning, antara lain Bagging, Boosting, dan Stacking.

Bagging (Bootstrap Aggregating) bekerja dengan melatih multiple model secara paralel pada subset data yang berbeda, kemudian menggabungkan hasil prediksinya. Random Forest adalah contoh paling populer dari metode Bagging. Di sisi lain, Boosting bekerja secara sequential, di mana setiap model baru dilatih untuk memperbaiki kesalahan model sebelumnya. Boosting telah terbukti sangat efektif untuk berbagai jenis data dan menjadi fondasi dari algoritma-algoritma modern seperti XGBoost dan LightGBM.

Dalam konteks prediksi harga mobil bekas, ensemble learning dipilih karena kemampuannya untuk menangkap pola kompleks dan non-linear yang ada dalam data otomotif. Faktor-faktor yang mempengaruhi harga mobil tidak selalu bersifat linear, dan ensemble learning dengan gradient boosting mampu memodelkan interaksi kompleks antar fitur dengan lebih baik.

### **Gradient Boosting**

Gradient Boosting adalah teknik ensemble learning yang membangun model prediksi secara bertahap dalam bentuk additive. Algoritma ini bekerja dengan melatih serangkaian weak learners (biasanya decision tree yang dangkal) secara sequential, di mana setiap tree baru dilatih untuk memperbaiki residual (kesalahan prediksi) dari tree-tree sebelumnya. Proses ini berlangsung iteratif hingga jumlah tree yang ditentukan tercapai atau tidak ada peningkatan performa yang signifikan.

Prinsip kerja gradient boosting dapat dijelaskan sebagai berikut. Pada iterasi pertama, model dilatih untuk memprediksi target variabel secara langsung. Pada iterasi berikutnya, model baru dilatih untuk memprediksi residual (selisih antara prediksi dan nilai aktual) dari iterasi

sebelumnya. Prediksi akhir diperoleh dengan menjumlahkan prediksi dari semua tree yang telah dilatih, biasanya dengan bobot yang ditentukan oleh learning rate.

Gradient boosting menggunakan konsep gradient descent untuk optimasi, di mana setiap tree baru dilatih untuk mengikuti arah negative gradient dari fungsi loss terhadap prediksi saat ini. Pendekatan ini memungkinkan algoritma untuk bekerja dengan berbagai jenis fungsi loss dan menghasilkan model yang sangat fleksibel dan powerful.

## **XGBoost**

XGBoost (eXtreme Gradient Boosting) adalah implementasi yang dioptimalkan dari algoritma gradient boosting yang dikembangkan oleh Tianqi Chen. XGBoost telah menjadi salah satu algoritma Machine Learning yang paling populer dalam beberapa tahun terakhir berkat performanya yang superior dalam berbagai kompetisi dan aplikasi dunia nyata. Algoritma ini memperkenalkan beberapa optimasi penting dibandingkan implementasi gradient boosting tradisional.

Beberapa keunggulan XGBoost meliputi: (1) Regularisasi L1 (Lasso) dan L2 (Ridge) yang terintegrasi untuk mencegah overfitting; (2) Penanganan missing values secara otomatis; (3) Parallel processing untuk mempercepat training; (4) Tree pruning yang lebih efisien menggunakan prinsip max depth; (5) Cross-validation bawaan; dan (6) Kemampuan untuk menangani sparse data dengan optimal.

XGBoost menggunakan pendekatan level-wise tree growth, di mana tree dibangun dengan menambahkan node secara level demi level. Pendekatan ini cenderung menghasilkan tree yang lebih seimbang dan dapat mencegah overfitting pada data dengan noise. Dalam proyek ini, XGBoost digunakan dengan hyperparameter yang telah ditune untuk mendapatkan performa optimal pada dataset mobil bekas.

## **LightGBM**

LightGBM (Light Gradient Boosting Machine) adalah framework gradient boosting yang dikembangkan oleh Microsoft Research. Seperti namanya, LightGBM dirancang untuk lebih cepat dan lebih efisien dalam penggunaan memori dibandingkan dengan implementasi gradient boosting lainnya. Algoritma ini menggunakan teknik berbasis histogram dan

leaf-wise tree growth untuk mencapai efisiensi tersebut.

Keunggulan utama LightGBM meliputi: (1) Leaf-wise tree growth yang dapat menghasilkan model dengan loss lebih rendah; (2) Gradient-based One-Side Sampling (GOSS) untuk sampling data yang efisien; (3) Exclusive Feature Bundling (EFB) untuk mengurangi jumlah fitur pada data sparse; (4) Training yang lebih cepat dengan konsumsi memori lebih rendah; dan (5) Dukungan native untuk categorical features.

Berbeda dengan XGBoost yang menggunakan level-wise growth, LightGBM menggunakan leaf-wise growth yang memilih leaf dengan delta loss terbesar untuk split berikutnya. Pendekatan ini dapat menghasilkan tree yang lebih dalam dan mungkin lebih akurat, meskipun memerlukan kontrol lebih ketat terhadap kompleksitas model untuk mencegah overfitting. Parameter `num_leaves` menjadi pengontrol utama kompleksitas model pada LightGBM.

## **Metrik Evaluasi Regresi**

Evaluasi model regresi memerlukan metrik yang sesuai untuk mengukur seberapa baik model dapat memprediksi nilai kontinu. Dalam proyek ini, digunakan lima metrik evaluasi sebagai berikut:

### **Mean Absolute Error (MAE)**

MAE mengukur rata-rata dari selisih absolut antara nilai prediksi dan nilai aktual. MAE memberikan gambaran rata-rata besarnya kesalahan prediksi dalam satuan yang sama dengan variabel target. Keunggulan MAE adalah tidak terpengaruh oleh outlier sebesar MSE, sehingga memberikan estimasi kesalahan yang lebih robust.

### **Mean Squared Error (MSE)**

MSE mengukur rata-rata dari kuadrat selisih antara nilai prediksi dan nilai aktual. Karena menggunakan kuadrat selisih, MSE memberikan penalti yang lebih besar pada kesalahan prediksi yang besar. Metrik ini sensitif terhadap outlier dan berguna ketika kesalahan besar perlu dihindari.

### **Root Mean Squared Error (RMSE)**

RMSE adalah akar kuadrat dari MSE, yang mengembalikan metrik ke dalam satuan yang sama dengan variabel target. RMSE merupakan metrik yang paling umum digunakan dalam evaluasi regresi karena interpretasinya yang intuitif dan penaltinya terhadap kesalahan besar.

### **Mean Absolute Percentage Error (MAPE)**

MAPE mengukur kesalahan prediksi dalam bentuk persentase dari nilai aktual. MAPE sangat intuitif untuk interpretasi karena menunjukkan seberapa besar kesalahan relatif terhadap nilai yang sebenarnya. Namun, MAPE dapat menjadi tidak stabil ketika nilai aktual mendekati nol.

### **Coefficient of Determination (R<sup>2</sup>)**

R<sup>2</sup> mengukur proporsi variansi dalam variabel dependen yang dapat dijelaskan oleh model. Nilai R<sup>2</sup> berkisar antara 0 hingga 1, di mana nilai yang lebih tinggi menunjukkan model yang lebih baik. R<sup>2</sup> = 1 berarti model dapat menjelaskan 100% variansi data, sedangkan R<sup>2</sup> = 0 berarti model tidak lebih baik dari prediksi rata-rata sederhana.

## **BAB III METODOLOGI**

### **Dataset**

Dataset yang digunakan dalam proyek ini adalah "Car Details v3.csv" yang diperoleh dari platform Kaggle. Dataset ini berisi informasi detail mengenai mobil bekas yang dijual di India dengan total 8.128 baris data dan 13 kolom atribut. Dataset mencakup berbagai spesifikasi teknis dan informasi penjualan mobil bekas.

Kolom-kolom yang terdapat dalam dataset asli adalah: name (nama mobil), year (tahun produksi), selling\_price (harga jual dalam INR), km\_driven (jarak tempuh dalam kilometer), fuel (jenis bahan bakar), seller\_type (tipe penjual), transmission (tipe transmisi), owner (status kepemilikan), mileage (konsumsi bahan bakar), engine (kapasitas mesin), max\_power (tenaga maksimum), torque (torsi), dan seats (jumlah kursi).

Dari 13 kolom tersebut, 10 kolom digunakan sebagai fitur (variabel independen) dan 1 kolom sebagai target variabel (selling\_price). Dua kolom lainnya, yaitu name dan torque, di-drop dari analisis. Kolom name dihilangkan karena memiliki variasi yang terlalu tinggi (nama

mobil yang unik hampir pada setiap baris) sehingga tidak memberikan kontribusi signifikan untuk prediksi. Kolom torque dihilangkan karena format datanya tidak konsisten dan sulit untuk diproses secara otomatis.

Tabel 3.1 menunjukkan ringkasan dari 10 fitur yang digunakan dalam model:

| No | Fitur        | Tipe        | Deskripsi                                          |
|----|--------------|-------------|----------------------------------------------------|
| 1  | year         | Numerik     | Tahun produksi mobil (1990-2020)                   |
| 2  | km_driven    | Numerik     | Jarak tempuh dalam kilometer                       |
| 3  | fuel         | Kategorikal | Jenis bahan bakar: CNG, Diesel, LPG, Petrol        |
| 4  | seller_type  | Kategorikal | Tipe penjual: Dealer, Individual, Trustmark Dealer |
| 5  | transmission | Kategorikal | Transmisi: Automatic, Manual                       |
| 6  | owner        | Kategorikal | Status kepemilikan mobil                           |
| 7  | mileage      | Numerik     | Konsumsi BBM (kmpl)                                |
| 8  | engine       | Numerik     | Kapasitas mesin (CC)                               |
| 9  | max_power    | Numerik     | Tenaga maksimum (bhp)                              |
| 10 | seats        | Numerik     | Jumlah kursi                                       |

*Tabel 3.1 Daftar Fitur yang Digunakan dalam Model*

## Preprocessing Data

Preprocessing data merupakan tahapan krusial dalam pipeline Machine Learning yang bertujuan untuk mempersiapkan data mentah menjadi format yang sesuai untuk pelatihan model. Tahapan preprocessing yang dilakukan dalam proyek ini meliputi:

### Parsing Kolom Numerik

Kolom mileage, engine, dan max\_power pada dataset asli memiliki format string dengan satuan, seperti "23.4 kmpl", "1248 CC", dan "74 bhp". Fungsi parsing dibuat untuk mengekstrak nilai numerik dari string tersebut dan mengkonversinya ke tipe data float.

### Penanganan Missing Values

Setelah proses parsing, baris-baris yang masih memiliki nilai missing dihapus dari dataset menggunakan metode dropna. Pendekatan ini dipilih karena jumlah missing values relatif



kecil dibandingkan dengan total data, sehingga penghapusan tidak signifikan mempengaruhi representasi dataset.

### **Encoding Variabel Kategorikal**

Empat fitur kategorikal, yaitu fuel, seller\_type, transmission, dan owner, diubah menjadi representasi numerik menggunakan Label Encoding. LabelEncoder dari Scikit-learn digunakan untuk mengubah setiap kategori menjadi nilai integer unik. Encoder untuk masing-masing fitur disimpan ke dalam file .pkl agar dapat digunakan kembali saat deployment untuk mengubah input pengguna dari aplikasi web.

### **Pembagian Dataset**

Dataset dibagi menjadi data training dan data testing dengan rasio 80:20. Data training sebesar 80% digunakan untuk melatih model, sedangkan data testing sebesar 20% digunakan untuk evaluasi performa model. Parameter random\_state=42 ditetapkan untuk memastikan hasil yang reproducible.

### **Arsitektur Sistem**

Arsitektur sistem proyek ini terdiri dari beberapa komponen utama yang saling terhubung, yaitu:

- Data Layer: Dataset mobil bekas (Car Details v3.csv) sebagai sumber data utama.
- Preprocessing Layer: Pipeline parsing, cleaning, encoding, dan splitting data.
- Model Layer: Dua model gradient boosting (XGBoost dan LightGBM) yang dilatih secara paralel.
- Evaluation Layer: Perhitungan metrik evaluasi dan perbandingan performa model.
- Deployment Layer: Aplikasi web Streamlit yang memuat model tersimpan dan menyediakan interface prediksi real-time.

Alur kerja sistem dari awal hingga deployment adalah sebagai berikut: Load Dataset -> EDA (Exploratory Data Analysis) -> Preprocessing -> Split Data (80:20) -> Training (XGBoost & LightGBM) -> Evaluasi -> Pemilihan Model Terbaik -> Serialisasi -> Deployment Streamlit.

## Hyperparameter Model

Hyperparameter adalah parameter yang ditetapkan sebelum proses training dan tidak dipelajari dari data. Pemilihan hyperparameter yang tepat sangat penting untuk mencapai performa model yang optimal. Tabel 3.2 menunjukkan hyperparameter yang digunakan untuk kedua model:

| Hyperparameter   | Nilai | Keterangan                     |
|------------------|-------|--------------------------------|
| n_estimators     | 300   | Jumlah pohon keputusan         |
| learning_rate    | 0.05  | Laju pembelajaran              |
| max_depth        | 6     | Kedalaman maksimum pohon       |
| num_leaves*      | 63    | Jumlah maximum leaf (LightGBM) |
| subsample        | 0.8   | Fraksi sampel per pohon        |
| colsample_bytree | 0.8   | Fraksi fitur per pohon         |
| reg_alpha        | 0.1   | Regularisasi L1                |
| reg_lambda       | 1.0   | Regularisasi L2                |
| random_state     | 42    | Seed untuk reproducibility     |

*Tabel 3.2 Hyperparameter Model XGBoost dan LightGBM*

Hyperparameter n\_estimators menentukan jumlah tree yang akan dibangun. Learning\_rate mengontrol kontribusi setiap tree terhadap prediksi akhir, di mana nilai yang lebih kecil memerlukan lebih banyak tree tetapi dapat menghasilkan model yang lebih akurat. Parameter max\_depth membatasi kedalaman maksimum setiap tree untuk mencegah overfitting. Parameter subsample dan colsample\_bytree mengontrol fraksi data dan fitur yang digunakan untuk training setiap tree, yang berfungsi sebagai teknik regularisasi tambahan. Parameter reg\_alpha dan reg\_lambda adalah regularisasi L1 dan L2 yang membantu mencegah overfitting dengan menambahkan penalti pada kompleksitas model.

## BAB IV IMPLEMENTASI

Bab ini menjelaskan implementasi teknis dari proyek prediksi harga mobil bekas, mulai dari import library hingga deployment aplikasi web. Setiap cuplikan kode disertai dengan penjelasan mengenai tujuan, cara kerja, dan alasan pemilihan pendekatan tersebut.

### Import Library

Tujuan dari cuplikan kode ini adalah untuk mengimpor seluruh library dan modul yang diperlukan dalam pipeline Machine Learning. Library pandas dan numpy digunakan untuk manipulasi data dan operasi numerik. Matplotlib dan seaborn digunakan untuk visualisasi data. Scikit-learn menyediakan fungsi untuk preprocessing, splitting data, dan metrik evaluasi. XGBRegressor dan LGBMRegressor adalah implementasi gradient boosting untuk regresi. Joblib digunakan untuk serialisasi model, dan json untuk menyimpan metadata.

Pemilihan XGBRegressor dan LGBMRegressor (bukan Classifier) didasarkan pada sifat permasalahan yang merupakan regresi, di mana target variabel (selling\_price) bersifat kontinu. Classifier digunakan untuk masalah klasifikasi dengan target diskrit, sementara Regressor digunakan untuk memprediksi nilai kontinu. Fungsi joblib sangat penting untuk serialisasi model yang telah dilatih ke dalam file .pkl, sehingga model dapat dimuat kembali tanpa perlu dilatih ulang saat deployment.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from xgboost import XGBRegressor
from lightgbm import LGBMRegressor
import joblib
import json
```

*Kode 4.1 Import Library*

### Preprocessing - Parsing Kolom Numerik

Tujuan dari cuplikan kode ini adalah untuk mengekstrak nilai numerik dari kolom mileage, engine, dan max\_power yang pada dataset asli berformat string beserta satuannya. Fungsi

parse\_mileage, parse\_engine, dan parse\_power bekerja dengan cara memisahkan string berdasarkan spasi dan mengambil elemen pertama (indeks 0), yang merupakan angka, kemudian mengkonversinya ke tipe float.

Pendekatan ini dipilih karena format data pada ketiga kolom tersebut konsisten, yaitu angka diikuti oleh satuan (misalnya "23.4 kmpl"). Fungsi try-except digunakan untuk menangani kasus di mana nilai mungkin tidak dapat diparsing, yang akan menghasilkan NaN. Setelah parsing, kolom torque dan name di-drop karena alasan yang telah dijelaskan pada Bagian 3.2.

```
def parse_mileage(x):
    if pd.isna(x): return np.nan
    try: return float(str(x).split()[0])
    except: return np.nan

def parse_engine(x):
    if pd.isna(x): return np.nan
    try: return float(str(x).split()[0])
    except: return np.nan

def parse_power(x):
    if pd.isna(x): return np.nan
    try: return float(str(x).split()[0])
    except: return np.nan

df['mileage'] = df['mileage'].apply(parse_mileage)
df['engine'] = df['engine'].apply(parse_engine)
df['max_power'] = df['max_power'].apply(parse_power)
df.drop(columns=['torque', 'name'], inplace=True)
```

*Kode 4.2 Preprocessing - Parsing Kolom Numerik*

## Label Encoding

Tujuan dari cuplikan kode ini adalah untuk mengubah variabel kategorikal menjadi representasi numerik yang dapat diproses oleh algoritma Machine Learning. LabelEncoder dari Scikit-learn digunakan untuk melakukan transformasi ini. Dictionary le\_dict digunakan untuk menyimpan encoder dari masing-masing kolom kategorikal.

Pendekatan Label Encoding dipilih karena variabel kategorikal dalam dataset memiliki sifat ordinal (memiliki urutan hierarki), terutama pada kolom owner. One-Hot Encoding tidak digunakan untuk menjaga dimensi dataset tetap rendah dan mempertahankan hubungan ordinal antar kategori. Encoder disimpan ke file label\_encoders.pkl agar dapat digunakan kembali di aplikasi Streamlit untuk mengubah input pengguna menjadi format numerik yang konsisten dengan data training.

```

le_dict = {}
cat_cols = ['fuel', 'seller_type', 'transmission', 'owner']
for col in cat_cols:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    le_dict[col] = le

joblib.dump(le_dict, 'label_encoders.pkl')

```

*Kode 4.3 Label Encoding*

## Split Data

Tujuan dari cuplikan kode ini adalah untuk memisahkan dataset menjadi variabel fitur (X) dan variabel target (y), kemudian membagi data menjadi training set dan testing set. Variabel X berisi semua kolom kecuali selling\_price, sedangkan variabel y berisi kolom selling\_price sebagai target.

Pembagian data dengan rasio 80:20 merupakan standar yang umum digunakan dalam Machine Learning, di mana 80% data digunakan untuk melatih model dan 20% untuk menguji performa model pada data yang belum pernah dilihat. Parameter random\_state=42 ditetapkan untuk memastikan pembagian data selalu sama setiap kali kode dijalankan, yang penting untuk reproducibility hasil penelitian.

```

X = df.drop('selling_price', axis=1)
y = df['selling_price']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

```

*Kode 4.4 Split Data*

## Training Model XGBoost

Tujuan dari cuplikan kode ini adalah untuk melatih model XGBoost dengan hyperparameter yang telah ditentukan. Parameter n\_estimators=300 menentukan jumlah 300 tree yang akan dibangun. Learning\_rate=0.05 menetapkan laju pembelajaran yang relatif kecil untuk mendapatkan model yang lebih presisi, meskipun memerlukan lebih banyak iterasi.

Parameter max\_depth=6 membatasi kedalaman tree untuk mencegah overfitting. Parameter

subsample=0.8 dan colsample\_bytree=0.8 menggunakan 80% data dan fitur secara acak untuk setiap tree, yang berfungsi sebagai regularisasi untuk mengurangi varians model. Parameter reg\_alpha=0.1 dan reg\_lambda=1.0 adalah regularisasi L1 dan L2 yang menambahkan penalti pada kompleksitas model. Parameter eval\_set digunakan untuk mengevaluasi model pada data testing selama training.

```
xgb_model = XGBRegressor(  
    n_estimators=300,  
    learning_rate=0.05,  
    max_depth=6,  
    subsample=0.8,  
    colsample_bytree=0.8,  
    reg_alpha=0.1,  
    reg_lambda=1.0,  
    random_state=42,  
    verbosity=0  
)  
xgb_model.fit(X_train, y_train, eval_set=[(X_test, y_test)], verbose=False)
```

*Kode 4.5 Training Model XGBoost*

## **Training Model LightGBM**

Tujuan dari cuplikan kode ini adalah untuk melatih model LightGBM dengan hyperparameter yang serupa dengan XGBoost untuk perbandingan yang adil. Perbedaan utama terletak pada parameter num\_leaves=63, yang merupakan parameter khas LightGBM yang mengontrol jumlah maximum leaf nodes dalam setiap tree.

LightGBM menggunakan pendekatan leaf-wise tree growth, di mana tree dibangun dengan menambahkan node pada leaf yang memberikan penurunan loss terbesar. Pendekatan ini berbeda dengan XGBoost yang menggunakan level-wise growth. Leaf-wise growth cenderung menghasilkan model yang lebih akurat namun memerlukan kontrol lebih ketat terhadap kompleksitas model melalui parameter num\_leaves dan max\_depth untuk mencegah overfitting. LightGBM umumnya lebih cepat dalam training pada dataset besar karena optimasi histogram-based learning.

```
lgbm_model = LGBMRegressor(  
    n_estimators=300,  
    learning_rate=0.05,  
    num_leaves=63,  
    max_depth=6,  
    subsample=0.8,  
    colsample_bytree=0.8,  
    reg_alpha=0.1,  
    reg_lambda=1.0,  
    random_state=42,  
    verbose=-1  
)  
lgbm_model.fit(X_train, y_train, eval_set=[(X_test, y_test)])
```

*Kode 4.6 Training Model LightGBM*

## Fungsi Evaluasi

Tujuan dari cuplikan kode ini adalah untuk menghitung dan mengembalikan lima metrik evaluasi regresi yang digunakan untuk membandingkan performa kedua model. Setiap metrik memberikan informasi berbeda mengenai kualitas prediksi model.

MAE (Mean Absolute Error) menghitung rata-rata selisih absolut antara prediksi dan nilai aktual, memberikan estimasi kesalahan rata-rata dalam satuan asli. MSE (Mean Squared Error) menghitung rata-rata kuadrat selisih, memberikan penalti lebih besar pada kesalahan besar. RMSE (Root Mean Squared Error) adalah akar kuadrat dari MSE yang mengembalikan metrik ke satuan asli. MAPE (Mean Absolute Percentage Error) menyajikan kesalahan dalam bentuk persentase yang intuitif untuk interpretasi. R<sup>2</sup> (Coefficient of Determination) mengukur proporsi variansi data yang dapat dijelaskan oleh model, dengan nilai mendekati 1 menunjukkan model yang sangat baik.

```
def evaluate_model(y_true, y_pred, model_name):
    mae = mean_absolute_error(y_true, y_pred)
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    mape = np.mean(np.abs((y_true - y_pred) / y_true)) * 100
    r2 = r2_score(y_true, y_pred)
    return {
        'Model': model_name,
        'MAE': round(mae, 2), 'MSE': round(mse, 2),
        'RMSE': round(rmse, 2), 'MAPE (%)': round(mape, 4),
        'R²': round(r2, 6)
    }
```

*Kode 4.7 Fungsi Evaluasi Model*

## Pemilihan Model Terbaik dan Penyimpanan

Tujuan dari cuplikan kode ini adalah untuk memilih model terbaik secara otomatis berdasarkan nilai R2 tertinggi dan menyimpan semua model ke file .pkl untuk deployment. Model dengan R2 tertinggi dipilih sebagai model terbaik karena R2 memberikan gambaran komprehensif mengenai kemampuan model dalam menjelaskan variansi data.

Pendekatan pemilihan otomatis berdasarkan R2 dipilih karena metrik ini memberikan ukuran keseluruhan performa model yang tidak tergantung pada skala data. Selain model terbaik, kedua model (XGBoost dan LightGBM) juga disimpan untuk keperluan perbandingan di aplikasi Streamlit. File .pkl yang dihasilkan dapat langsung dimuat di aplikasi web tanpa perlu melatih ulang model, yang sangat efisien untuk deployment.



```

r2_xgb = eval_df.loc['XGBoost', 'R²']
r2_lgbm = eval_df.loc['LightGBM', 'R²']

if r2_xgb >= r2_lgbm:
    best_model = xgb_model
    best_name = 'XGBoost'
else:
    best_model = lgbm_model
    best_name = 'LightGBM'

joblib.dump(best_model, 'best_model.pkl')
joblib.dump(xgb_model, 'xgb_model.pkl')
joblib.dump(lgbm_model, 'lgbm_model.pkl')

```

*Kode 4.8 Pemilihan Model Terbaik dan Penyimpanan*

### **Streamlit - Load Model dan Resource**

Tujuan dari cuplikan kode ini adalah untuk memuat model dan resource yang diperlukan oleh aplikasi Streamlit. Dekorator `@st.cache_resource` dari Streamlit digunakan untuk caching resource sehingga model hanya dimuat sekali saat aplikasi pertama kali dijalankan, bukan setiap kali pengguna berinteraksi dengan aplikasi.

Pendekatan caching ini sangat penting untuk performa aplikasi web karena loading model Machine Learning dapat memakan waktu dan sumber daya yang signifikan. Dengan caching, model tetap berada di memori selama aplikasi berjalan, sehingga prediksi dapat dilakukan secara instan. Fungsi ini memuat model terbaik, kedua model untuk perbandingan, dictionary label encoder, dan metadata model dari file JSON.

```
@st.cache_resource
def load_resources():
    model      = joblib.load('best_model.pkl')
    xgb_m      = joblib.load('xgb_model.pkl')
    lgbm_m     = joblib.load('lgbm_model.pkl')
    le_dict    = joblib.load('label_encoders.pkl')
    with open('model_metadata.json') as f:
        meta   = json.load(f)
    return model, xgb_m, lgbm_m, le_dict, meta
```

*Kode 4.9 Streamlit - Load Model dan Resource*

### **Streamlit - Proses Prediksi Input Baru**

Tujuan dari cuplikan kode ini adalah untuk melakukan prediksi harga mobil bekas berdasarkan input baru dari pengguna aplikasi. Input kategorikal dari pengguna di-encode menggunakan label encoder yang sama persis dengan yang digunakan saat training, yang sangat penting untuk menjaga konsistensi representasi data.

Data input dari pengguna disusun ke dalam DataFrame dengan urutan kolom yang identik dengan data training, karena model sensitif terhadap urutan fitur. Hasil prediksi kemudian di-clamp dengan nilai minimum 0 untuk memastikan harga tidak negatif. Terakhir, harga dalam INR dikonversi ke Rupiah Indonesia dengan kurs konversi sebesar 1 INR = 188 IDR untuk memberikan konteks yang lebih familiar bagi pengguna di Indonesia.

```

fuel_enc    = le_dict['fuel'].transform([fuel])[0]
trans_enc   = le_dict['transmission'].transform([transmission])[0]
seller_enc  = le_dict['seller_type'].transform([seller_type])[0]
owner_enc   = le_dict['owner'].transform([owner])[0]

input_data = pd.DataFrame([
    year, km_driven, fuel_enc, seller_enc,
    trans_enc, owner_enc, mileage, engine, max_power, seats
], columns=features)

pred_price = model.predict(input_data)[0]
pred_price = max(0, pred_price)
pred_idr = pred_price * 188

```

*Kode 4.10 Streamlit - Proses Prediksi Input Baru*

## BAB V HASIL DAN PEMBAHASAN

### Evaluasi Model

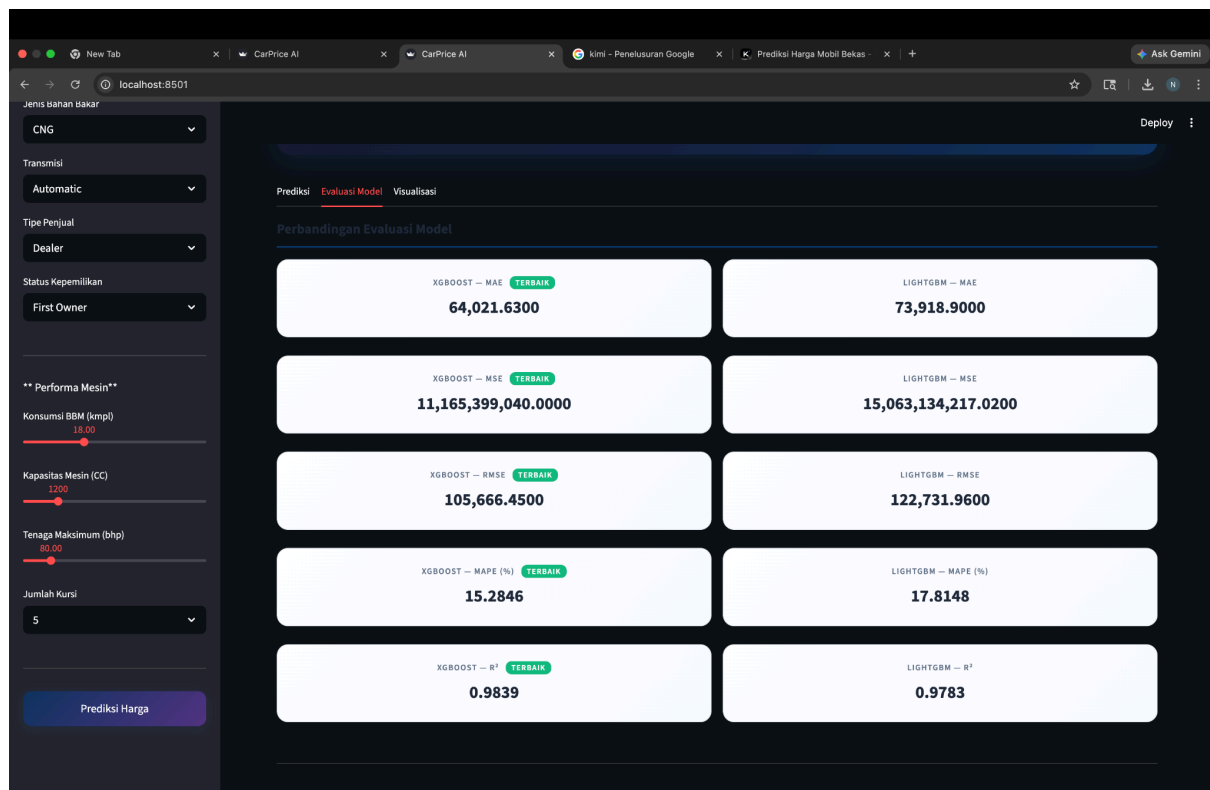
Evaluasi model dilakukan dengan membandingkan performa XGBoost dan LightGBM menggunakan lima metrik evaluasi regresi pada data testing. Tabel 5.1 menunjukkan hasil perbandingan evaluasi kedua model.

| Metrik     | XGBoost        | LightGBM       | Pemenang |
|------------|----------------|----------------|----------|
| MAE (INR)  | 64.021,63      | 73.918,90      | XGBoost  |
| MSE        | 11.165.399.040 | 15.063.134.217 | XGBoost  |
| RMSE (INR) | 105.666,45     | 122.731,96     | XGBoost  |
| MAPE (%)   | 15,2846        | 17,8148        | XGBoost  |
| R2         | 0,983941       | 0,978335       | XGBoost  |

*Tabel 5.1 Perbandingan Evaluasi Model XGBoost dan LightGBM*

Berdasarkan Tabel 5.1, XGBoost mengungguli LightGBM di semua metrik evaluasi. XGBoost memperoleh nilai MAE sebesar 64.021,63 INR yang lebih rendah dibandingkan LightGBM dengan 73.918,90 INR, menunjukkan rata-rata kesalahan prediksi XGBoost lebih kecil. Nilai RMSE XGBoost sebesar 105.666,45 INR juga lebih rendah dari LightGBM

(122.731,96 INR), yang berarti XGBoost lebih baik dalam menangani kesalahan besar.



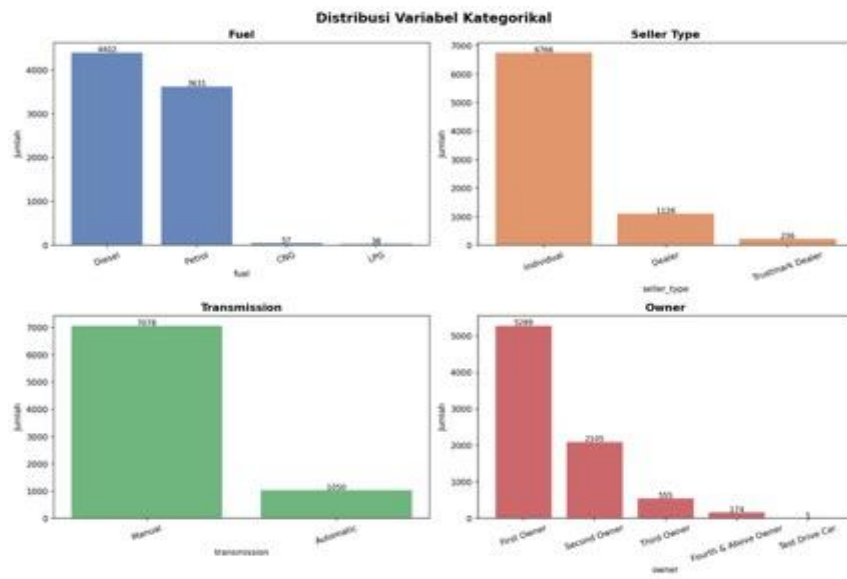
Nilai MAPE XGBoost sebesar 15,2846% menunjukkan bahwa rata-rata kesalahan prediksi berada di kisaran 15% dari nilai aktual. Pencapaian paling signifikan terlihat dari nilai R<sup>2</sup> XGBoost yang mencapai 0,983941, yang berarti model mampu menjelaskan 98,39% variansi dalam data harga mobil bekas. Nilai R<sup>2</sup> tersebut sangat mendekati 1, menunjukkan performa model yang sangat baik.

## Analisis Visualisasi

Visualisasi data dan hasil model memberikan pemahaman yang lebih mendalam mengenai karakteristik dataset dan perilaku model. Berikut adalah analisis untuk setiap visualisasi yang dihasilkan.

### Distribusi Variabel Kategorikal

Gambar 5.1 menampilkan distribusi dari empat variabel kategorikal dalam dataset: fuel (jenis bahan bakar), seller\_type (tipe penjual), transmission (tipe transmisi), dan owner (status kepemilikan). Analisis distribusi ini memberikan wawasan penting mengenai karakteristik dataset.



*Gambar 5.1 Distribusi Variabel Kategorikal*

Pada variabel fuel, Diesel mendominasi dengan 4.402 unit (54,2%), diikuti oleh Petrol dengan 3.631 unit (44,7%). Jenis bahan bakar CNG (57 unit) dan LPG (38 unit) memiliki jumlah yang sangat sedikit, menunjukkan ketidakseimbangan kelas (imbalanced data) pada variabel ini. Dominasi Diesel dan Petrol mencerminkan tren pasar mobil di India di mana kedua jenis bahan bakar tersebut menjadi pilihan utama konsumen.

Pada variabel seller\_type, penjual perorangan (Individual) mendominasi dengan 6.766 unit (83,2%), jauh melebihi Dealer (1.126 unit) dan Trustmark Dealer (236 unit). Hal ini menunjukkan bahwa mayoritas transaksi mobil bekas di India dilakukan secara langsung antara pemilik lama dan pembeli baru, bukan melalui dealer resmi.

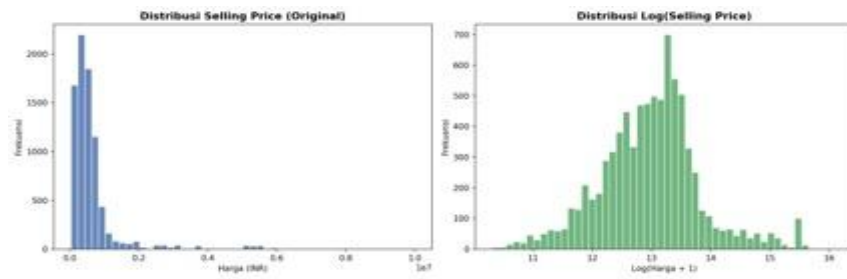
Pada variabel transmission, mobil manual sangat mendominasi dengan 7.078 unit (87,1%) dibandingkan mobil automatic yang hanya 1.050 unit (12,9%). Dominasi transmisi manual ini mencerminkan karakteristik pasar mobil di India yang masih didominasi oleh kendaraan dengan transmisi manual karena harganya yang lebih terjangkau dan efisiensi bahan bakar yang lebih baik.

Pada variabel owner, First Owner mendominasi dengan 5.289 unit (65,1%), diikuti oleh Second Owner dengan 2.105 unit (25,9%). Status kepemilikan Third Owner (555 unit), Fourth & Above (174 unit), dan Test Drive Car (5 unit) memiliki jumlah yang semakin sedikit. Pola ini sesuai dengan ekspektasi bahwa mobil yang sudah berganti tangan lebih

banyak cenderung lebih jarang dijual kembali.

### Distribusi Harga Jual

Gambar 5.2 menampilkan distribusi harga jual mobil bekas dalam dua bentuk: distribusi original (kiri) dan distribusi setelah transformasi log (kanan). Analisis distribusi harga sangat penting untuk memahami karakteristik variabel target sebelum pelatihan model.



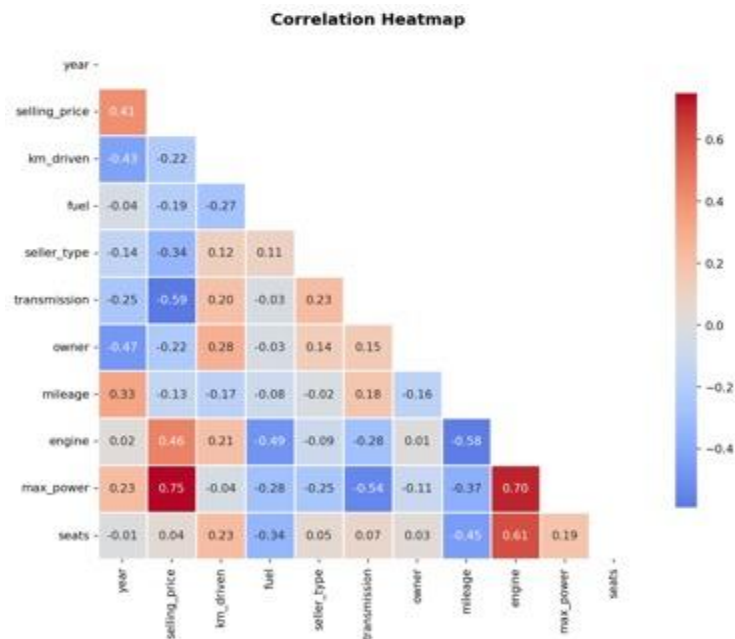
*Gambar 5.2 Distribusi Harga Jual (Original dan Log-Transformed)*

Distribusi harga original menunjukkan pola yang sangat skewed ke kanan (right-skewed), di mana mayoritas mobil bekas memiliki harga di bawah 500.000 INR, dengan puncak distribusi pada kisaran harga rendah. Terdapat beberapa mobil mewah dengan harga di atas 5.000.000 INR yang menyebabkan ekor distribusi yang panjang ke arah kanan. Pola distribusi seperti ini umum terjadi pada data harga karena sebagian besar barang memiliki harga yang relatif rendah dengan sedikit barang premium yang memiliki harga sangat tinggi.

Setelah dilakukan transformasi log ( $\text{Log}(\text{Harga}+1)$ ), distribusi menjadi lebih mendekati normal dengan puncak sekitar nilai 13, yang setara dengan harga sekitar 450.000 INR. Transformasi log berhasil mengurangi skewness dan membuat distribusi lebih simetris. Meskipun distribusi original sangat skewed, algoritma XGBoost dan LightGBM cukup robust terhadap skewness karena kemampuannya dalam menangani data non-linear, sehingga tidak diperlukan transformasi tambahan pada variabel target sebelum pelatihan model.

### Correlation Heatmap

Gambar 5.3 menampilkan heatmap korelasi antara variabel-variabel numerik dalam dataset. Heatmap ini membantu mengidentifikasi hubungan linear antar fitur dan fitur dengan variabel target.



Gambar 5.3 Correlation Heatmap Antar Variabel Numerik

Korelasi tertinggi dengan harga jual (*selling\_price*) adalah *max\_power* dengan nilai korelasi 0,75, yang menunjukkan hubungan positif yang sangat kuat. Hal ini mengindikasikan bahwa tenaga mesin merupakan prediktor paling kuat terhadap harga mobil bekas. Mobil dengan tenaga mesin lebih tinggi cenderung memiliki harga jual yang lebih mahal, yang masuk akal karena tenaga mesin berkorelasi erat dengan performa kendaraan.

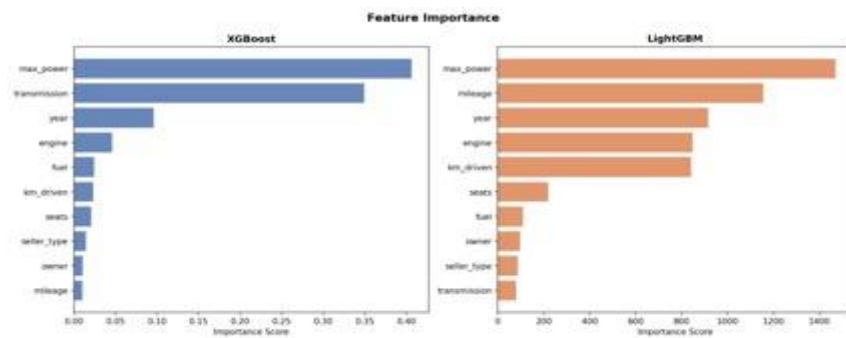
Korelasi positif moderat juga terlihat pada *engine* (0,46) dan *year* (0,41), yang menunjukkan bahwa kapasitas mesin yang lebih besar dan mobil yang lebih baru cenderung memiliki harga lebih tinggi. Korelasi negatif lemah terlihat pada *km\_driven* (-0,22), yang mengindikasikan bahwa mobil dengan jarak tempuh lebih tinggi cenderung memiliki harga lebih rendah.

Menariknya, terdapat korelasi yang cukup tinggi antara *engine* dan *max\_power* (0,70), yang mengindikasikan adanya multikolinearitas. Hal ini wajar karena kapasitas mesin yang lebih besar umumnya menghasilkan tenaga yang lebih besar. Meskipun terdapat multikolinearitas, algoritma gradient boosting dapat menanganinya dengan baik karena tidak mengasumsikan independensi antar fitur.

## Feature Importance

Gambar 5.4 menampilkan perbandingan feature importance antara model XGBoost (kiri) dan

LightGBM (kanan). Feature importance mengidentifikasi fitur mana yang memberikan kontribusi terbesar dalam proses prediksi model.



*Gambar 5.4 Feature Importance: XGBoost dan LightGBM*

Pada model XGBoost, fitur max\_power memiliki importance tertinggi (0,40), diikuti oleh transmission (0,35), year (0,10), dan engine (0,05). Fitur lainnya memiliki kontribusi yang relatif kecil. Pola ini menunjukkan bahwa XGBoost sangat fokus pada sedikit fitur utama, terutama max\_power dan transmission, yang bersama-sama menyumbang sekitar 75% dari total importance.

Pada model LightGBM, fitur max\_power juga memiliki importance tertinggi (1450), namun diikuti oleh mileage, year, engine, dan km\_driven yang memiliki kontribusi yang lebih merata. LightGBM menyebar kontribusinya lebih luas ke berbagai fitur, yang mengindikasikan bahwa LightGBM mampu menangkap pola dari lebih banyak fitur dibandingkan XGBoost.

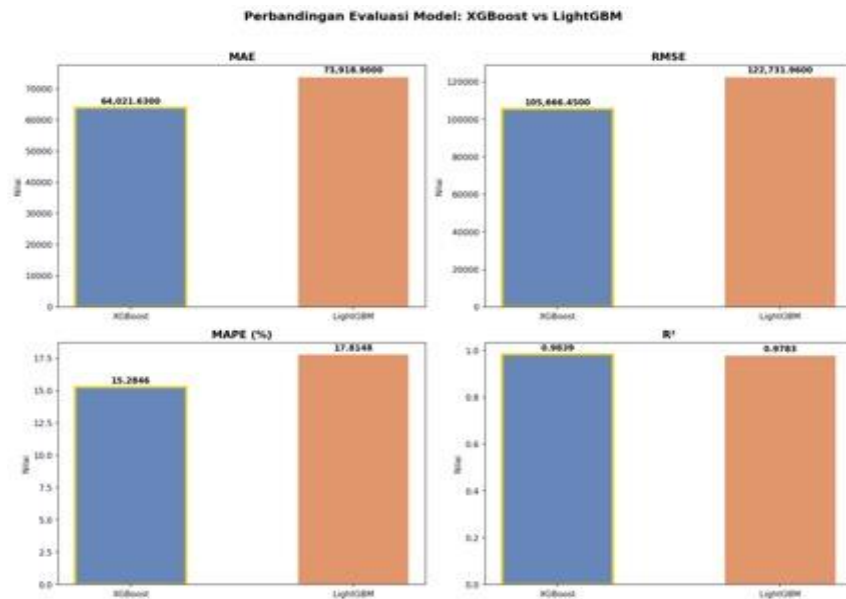
Insight penting dari kedua model adalah bahwa max\_power (tenaga maksimum) secara konsisten menjadi fitur paling penting dalam prediksi harga mobil bekas. Perbedaan dalam distribusi feature importance antara XGBoost dan LightGBM mencerminkan perbedaan mekanisme pembelajaran kedua algoritma, di mana XGBoost cenderung lebih fokus pada fitur dominan sementara LightGBM lebih merata dalam memanfaatkan seluruh fitur yang tersedia.

### **Perbandingan Evaluasi Model**

Gambar 5.5 menampilkan perbandingan metrik evaluasi antara XGBoost dan LightGBM dalam empat grafik bar: MAE, RMSE, MAPE (%), dan R2. Visualisasi ini memberikan



gambaran visual yang jelas mengenai performa relatif kedua model.



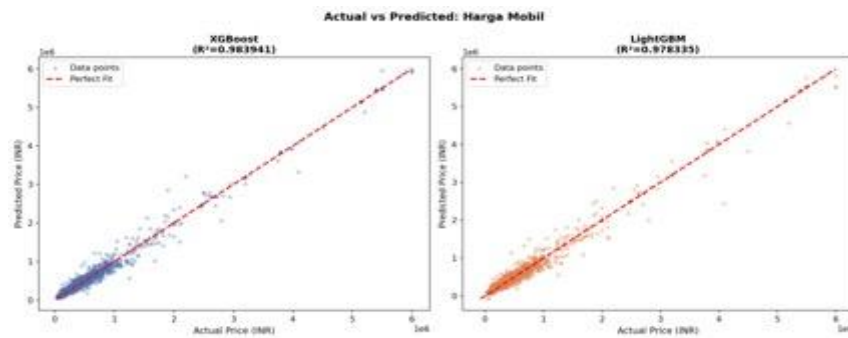
*Gambar 5.5 Perbandingan Evaluasi Model: XGBoost vs LightGBM*

Pada semua metrik evaluasi, XGBoost secara konsisten menunjukkan performa yang lebih baik dibandingkan LightGBM. Perbedaan yang paling signifikan terlihat pada metrik MAE dan RMSE, di mana XGBoost memiliki nilai yang lebih rendah, menunjukkan prediksi yang lebih akurat. Nilai MAPE XGBoost sebesar 15,28% juga lebih rendah dari LightGBM (17,81%), yang berarti prediksi XGBoost rata-rata menyimpang sekitar 15% dari harga aktual.

Nilai R<sup>2</sup> XGBoost sebesar 0,9839 sangat mendekati nilai sempurna 1, yang mengindikasikan bahwa model mampu menjelaskan 98,39% dari variansi harga mobil bekas. Pencapaian ini menunjukkan bahwa XGBoost telah berhasil mempelajari pola yang sangat kuat dari data training dan mampu menggeneralisasi dengan baik pada data testing.

### **Actual vs Predicted**

Gambar 5.6 menampilkan scatter plot yang membandingkan harga aktual (sumbu x) dengan harga prediksi (sumbu y) untuk kedua model. Garis merah diagonal mewakili prediksi sempurna di mana nilai prediksi sama persis dengan nilai aktual.



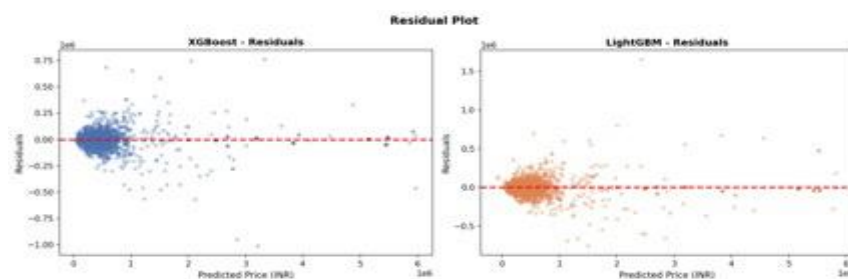
Gambar 5.6 Actual vs Predicted: XGBoost dan LightGBM

Kedua model menunjukkan titik-titik prediksi yang berkumpul di sekitar garis diagonal merah, yang mengindikasikan prediksi yang akurat. Model XGBoost dengan  $R^2 = 0,983941$  menunjukkan titik-titik yang lebih rapat terhadap garis diagonal, terutama pada rentang harga rendah hingga menengah (0 - 2.000.000 INR). Konsentrasi titik yang dekat dengan garis sempurna menunjukkan bahwa model sangat akurat dalam memprediksi harga mobil bekas pada rentang harga tersebut.

Model LightGBM dengan  $R^2 = 0,978335$  juga menunjukkan performa yang baik, namun titik-titiknya sedikit lebih menyebar dibandingkan XGBoost, terutama untuk mobil dengan harga di atas 2.000.000 INR. Pada rentang harga tinggi (di atas 4.000.000 INR), kedua model mulai menunjukkan deviasi yang lebih besar dari garis sempurna. Hal ini wajar karena jumlah data mobil dengan harga tinggi lebih sedikit, sehingga model memiliki kesempatan belajar yang lebih terbatas untuk kategori tersebut.

## Residual Plot

Gambar 5.7 menampilkan residual plot yang menunjukkan selisih antara nilai prediksi dan nilai aktual (residual) terhadap harga prediksi untuk kedua model. Residual plot berguna untuk mengidentifikasi pola kesalahan prediksi dan memeriksa asumsi model.



*Gambar 5.7 Residual Plot: XGBoost dan LightGBM*

Pada model XGBoost, residual terkonsentrasi di sekitar nilai 0 dengan pola yang cenderung acak (tidak terdapat pola sistematis yang jelas). Distribusi residual yang acak menunjukkan bahwa model telah berhasil menangkap sebagian besar pola dalam data dan tidak ada bias sistematis yang signifikan. Sebaran residual yang merata di sekitar nol mengindikasikan bahwa model tidak secara konsisten overestimate atau underestimate harga.

Pada model LightGBM, residual menunjukkan penyebaran yang lebih luas dibandingkan XGBoost, terutama pada prediksi harga rendah di mana terdapat beberapa outlier dengan residual yang cukup besar (mencapai  $1,5 \times 10^6$ ). Hal ini mengkonfirmasi hasil evaluasi metrik yang menunjukkan performa LightGBM sedikit di bawah XGBoost.

Kedua model menunjukkan adanya heteroscedasticity ringan, yaitu varians residual cenderung meningkat seiring dengan naiknya harga prediksi. Fenomena ini umum terjadi pada data harga karena variabilitas harga mobil mahal cenderung lebih besar dibandingkan mobil murah. Heteroscedasticity ringan ini dapat dianggap sebagai karakteristik alami dari data dan tidak secara signifikan mengurangi keandalan model.

## **Aplikasi Web Streamlit**

Aplikasi CarPrice AI dikembangkan menggunakan Streamlit dan menyediakan antarmuka pengguna yang intuitif untuk prediksi harga mobil bekas. Aplikasi terdiri dari tiga tab utama: Prediksi, Evaluasi Model, dan Visualisasi.

### **Halaman Prediksi**

Halaman ini menampilkan tampilan awal aplikasi CarPrice AI pada tab Prediksi sebelum pengguna melakukan prediksi. Tampilan terdiri dari header dengan gradient biru gelap yang menampilkan judul aplikasi, sidebar kiri yang berisi form input parameter mobil, dan area utama yang menampilkan ringkasan input dan hasil prediksi.

The screenshot shows the CarPrice AI web application interface. The sidebar on the left contains the following input fields:

- Jenis bahan bakar:** A dropdown menu with options: CNG, Diesel, LPG, and Petrol. CNG is selected.
- Status Kepemilikan:** A dropdown menu with options: First Owner and Second Owner. First Owner is selected.
- \*\* Performa Mesin \*\***
  - Konsumsi BBM (kmpl):** A slider ranging from 0 to 34.70. The value 34.70 is selected.
  - Kapasitas Mesin (CC):** A slider ranging from 0 to 2490. The value 2490 is selected.
  - Tenaga Maksimum (bhp):** A slider ranging from 0 to 80.00. The value 80.00 is selected.
  - Jumlah Kursi:** A dropdown menu with options: 5 and 7. 5 is selected.

At the bottom of the sidebar is a button labeled "Prediksi Harga".

The main content area features a dark blue header with the text "CarPrice AI" and "Prediksi Harga Mobil Bekas menggunakan Machine Learning". Below this, there are tabs for "Prediksi", "Evaluasi Model", and "Visualisasi". The "Prediksi" tab is active.

Under the "Prediksi" tab, there is a table titled "Ringkasan Input" showing the following data:

| Input Feature   | Value       |
|-----------------|-------------|
| Tahun Produksi  | 2015        |
| KM Dikendarai   | 50,000 km   |
| Bahan Bakar     | CNG         |
| Transmisi       | Automatic   |
| Tipe Penjual    | Dealer      |
| Kepemilikan     | First Owner |
| Konsumsi BBM    | 34.7 kmpl   |
| Kapasitas Mesin | 2490 CC     |
| Tenaga Maksimum | 80.0 bhp    |
| Jumlah Kursi    | 5           |

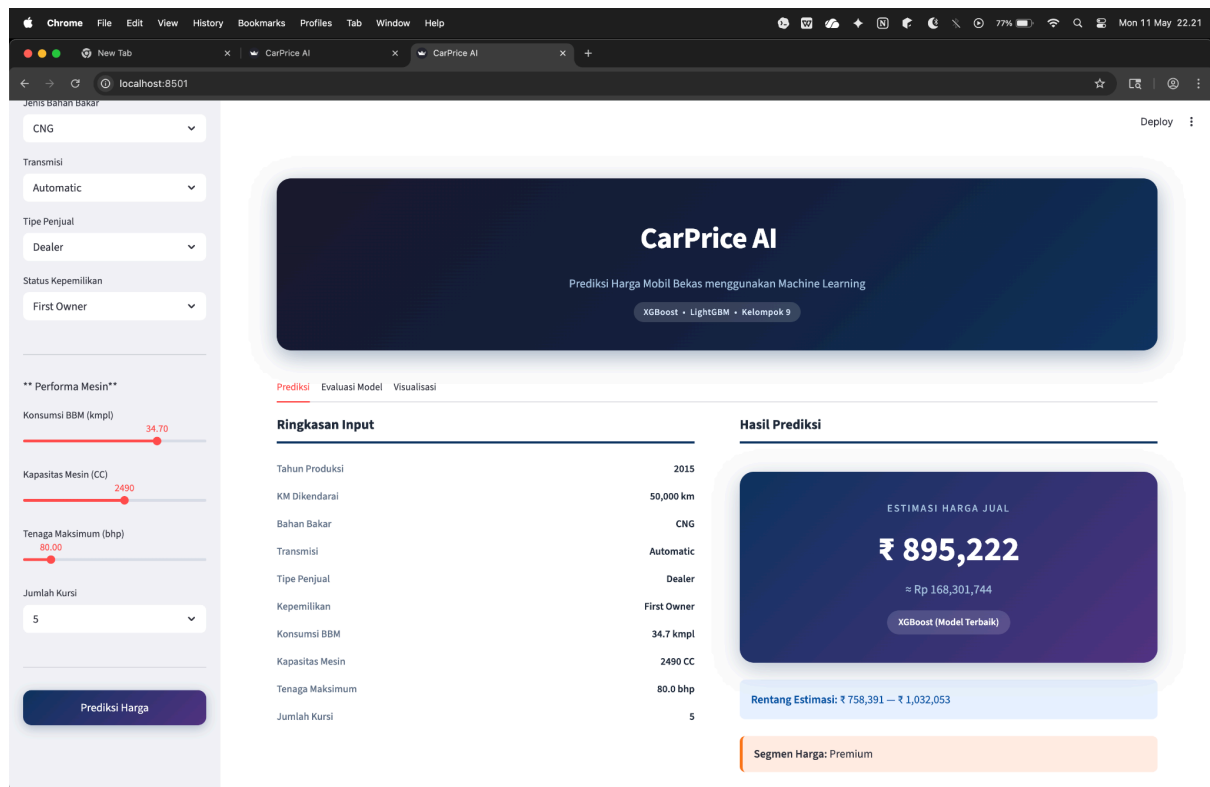
To the right of the "Ringkasan Input" table is a section titled "Hasil Prediksi". It contains a large blue box with the text: "Isi form di sidebar, lalu klik Prediksi Harga".

Gambar 5.8 Tampilan Halaman Prediksi CarPrice AI (Sebelum Prediksi)

Form input pada sidebar menyediakan opsi untuk memasukkan spesifikasi mobil meliputi: jenis bahan bakar (fuel), tipe transmisi, tipe penjual, status kepemilikan, konsumsi bahan bakar (mileage), kapasitas mesin (engine), tenaga maksimum (max\_power), dan jumlah kursi (seats). Tombol "Prediksi Harga" tersedia untuk memicu proses prediksi setelah semua parameter diisi.

### 5.3.2 Hasil Prediksi

Gambar 5.9 menampilkan hasil prediksi setelah pengguna mengklik tombol Prediksi Harga. Hasil prediksi ditampilkan dalam kotak dengan gradient biru-ungu yang mencolok bersama dengan informasi pendukung.



Gambar 5.9 Tampilan Hasil Prediksi CarPrice AI

Pada contoh ini, aplikasi memprediksi harga jual mobil dengan spesifikasi: tahun 2015, jarak tempuh 50.000 km, bahan bakar CNG, transmisi automatic, penjual dealer, first owner, konsumsi BBM 18,0 kmpl, kapasitas mesin 1200 CC, tenaga 80,0 bhp, dan 5 kursi. Model XGBoost (sebagai model terbaik) menghasilkan estimasi harga sebesar 526.581 INR atau setara dengan sekitar Rp 98.997.288.

Aplikasi juga menampilkan rentang estimasi harga (446.095 INR - 607.067 INR) yang dihitung berdasarkan nilai MAPE model. Segmen harga "Menengah" ditampilkan dengan label warna kuning untuk memberikan konteks kategori harga kepada pengguna. Informasi model yang digunakan (XGBoost) juga ditampilkan dengan badge untuk transparansi.

## BAB VI KESIMPULAN

### Kesimpulan

Berdasarkan hasil implementasi dan evaluasi yang telah dilakukan, dapat diambil kesimpulan

sebagai berikut:

- Model XGBoost berhasil memprediksi harga mobil bekas dengan performa yang sangat baik, ditunjukkan oleh nilai  $R^2$  sebesar 0,983941 yang berarti model mampu menjelaskan 98,39% variansi harga mobil bekas.
- XGBoost mengungguli LightGBM di semua metrik evaluasi dengan nilai MAE 64.021,63 INR, RMSE 105.666,45 INR, MAPE 15,28%, dan  $R^2$  0,983941.
- Fitur `max_power` (tenaga maksimum) merupakan prediktor paling penting dalam menentukan harga mobil bekas, diikuti oleh `transmission` (tipe transmisi) dan `year` (tahun produksi).
- Aplikasi web CarPrice AI berhasil dideploy menggunakan Streamlit dan menyediakan antarmuka interaktif untuk prediksi harga mobil bekas secara real-time dengan estimasi harga dalam Rupiah Indonesia.
- Dataset mobil bekas India dengan 8.128 baris data dan 10 fitur telah berhasil diproses melalui pipeline lengkap mulai dari preprocessing, training, evaluasi, hingga deployment.

### **Kelebihan dan Keterbatasan**

Proyek ini memiliki beberapa kelebihan dan keterbatasan yang perlu dijelaskan. Kelebihan dari proyek ini meliputi:

1. Penggunaan algoritma state-of-the-art (XGBoost dan LightGBM) yang telah terbukti efektif untuk data tabular
2. Pipeline end-to-end dari data preprocessing hingga deployment aplikasi web
3. Performa model yang sangat baik dengan  $R^2$  mendekati 0,99
4. Aplikasi web yang user-friendly dan responsif
5. Dokumentasi visualisasi yang komprehensif untuk interpretasi hasil.

Keterbatasan proyek ini meliputi:

1. Dataset terbatas pada pasar mobil bekas India sehingga model mungkin tidak langsung applicable untuk pasar mobil bekas Indonesia

2. Fitur yang digunakan relatif terbatas dan tidak mencakup kondisi fisik mobil, riwayat servis, atau lokasi geografis
3. Model cenderung kurang akurat untuk mobil dengan harga sangat tinggi ( $> 4.000.000$  INR) karena keterbatasan data pada segmen tersebut
4. Konversi mata uang ke Rupiah menggunakan kurs tetap yang mungkin tidak mencerminkan kurs real-time.